



D Data
S Structures
A Algorithms



Data Structures & Algorithms

Today's Topic: Dynamic Programming



Dr. Si Thu Aung



in/drsithuaung/



<https://sithuaung-ink.github.io/data-analyst-portfolio/>

5/14/2026

What is Dynamic Programming?

Dynamic Programming (DP) is an algorithmic technique for solving an optimization problem by breaking it down into simpler subproblems and using the fact that the optimal solution to the overall problem depends on the optimal solutions to its subproblems.

Example: 1

$$\boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} = \boxed{7}$$

$$\boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{1} + \boxed{2} = \boxed{9}$$

└──┘
7

What is Dynamic Programming?

Optimal Substructure:

If the overall optimal solution to a problem can be constructed from the optimal solutions of its subproblems, then the problem has optimal substructure.

Example: $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

Overlapping Subproblem:

Subproblems are smaller versions of the original problem. Any problem has overlapping subproblems if finding its solution involves the same sub-problem multiple times.



Top Down with Memoization

Solve the bigger problem by recursively solving smaller subproblems. Whenever we solve a subproblem, we cache its result, so we don't end up solving it repeatedly when it's called multiple times. This technique of storing the results of already solved subproblems is called **Memoization**.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$$

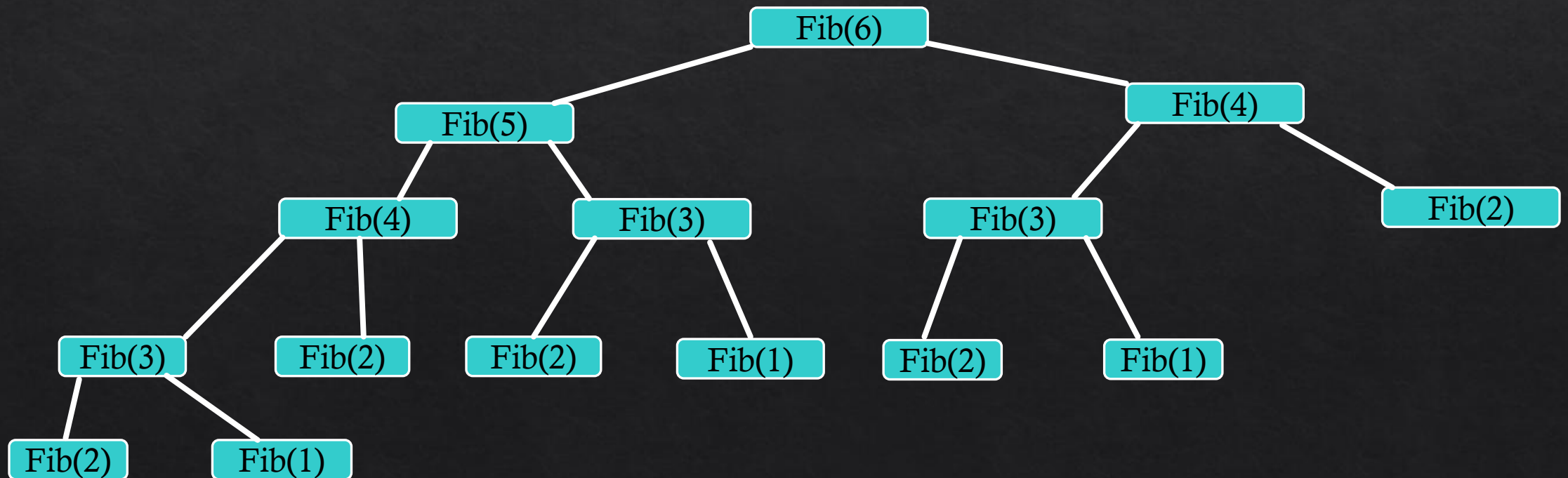
```
Fibonacci(n):  
    if n < 1 return error message  
    if n = 1 return 0  
    if n = 2 return 1  
    else:  
        return Fibonacci(n-1) + Fibonacci(n-2)
```


Top Down with Memoization

Solve the bigger problem by recursively solving smaller subproblems. Whenever we solve a subproblem, we cache its result, so we don't end up solving it repeatedly when it's called multiple times. This technique of storing the results of already solved subproblems is called Memoization.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$$



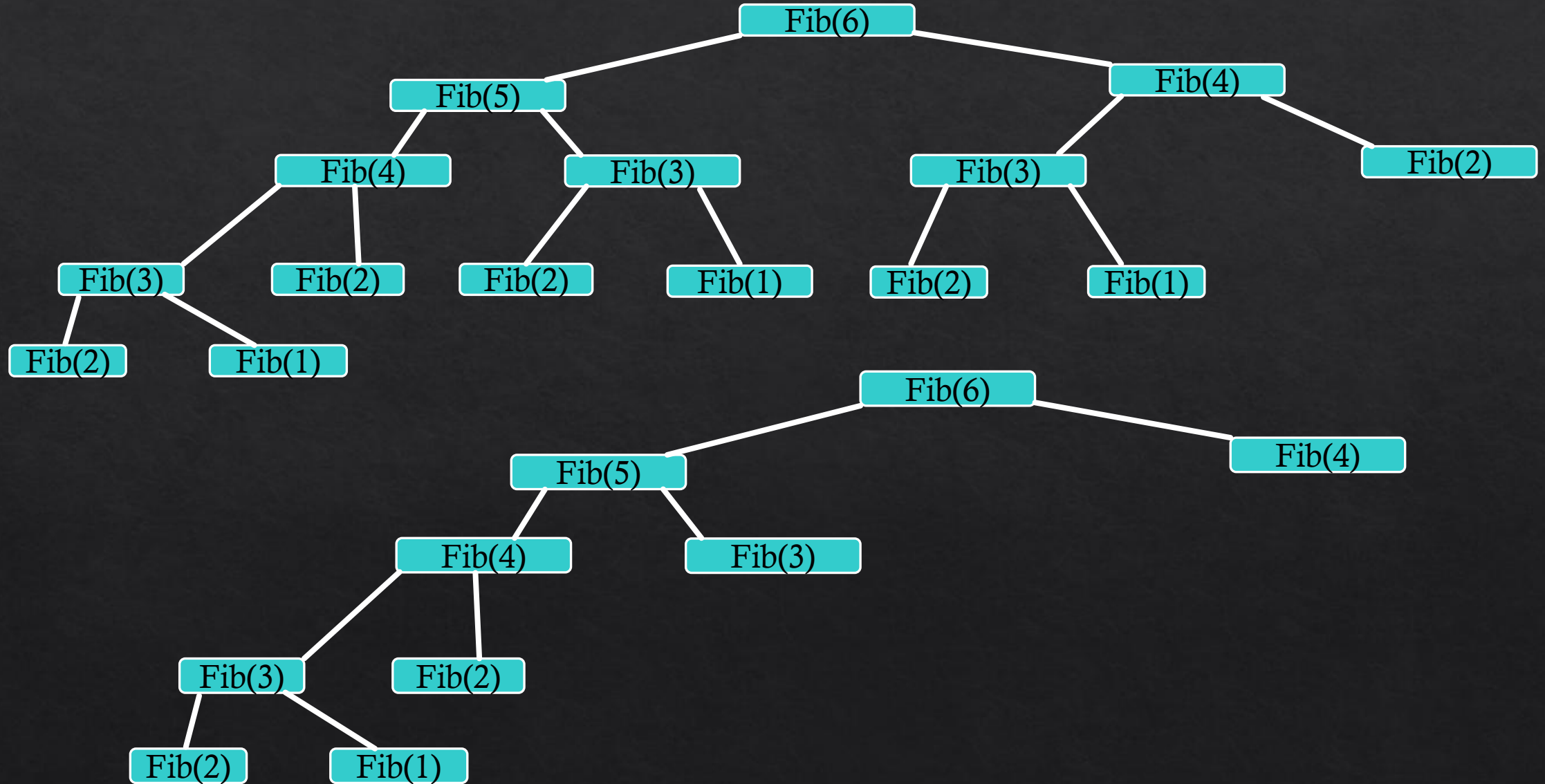
Top Down with Memoization

Solve the bigger problem by recursively solving smaller subproblems. Whenever we solve a subproblem, we cache its result, so we don't end up solving it repeatedly when it's called multiple times. This technique of storing the results of already solved subproblems is called Memoization.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

```
Fibonacci(n):  
    if n < 1 return error message  
    if n = 1 return 0  
    if n = 2 return 1  
    if not n in memo:  
        memo[n] = Fibonacci(n-1, memo) + Fibonacci(n-2, memo)  
    return memo[n]
```

Top Down with Memoization



Top Down with Memoization

Solve the bigger problem by recursively solving smaller subproblems. Whenever we solve a subproblem, we cache its result, so we don't end up solving it repeatedly when it's called multiple times. This technique of storing the results of already solved subproblems is called Memoization.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$$

F1	F2	F3	F4	F5	F6
0	1				F4+F5

F1	F2	F3	F4	F5	F6
0	1			F3+F4	F4+F5

F1	F2	F3	F4	F5	F6
0	1		F2+F3	F3+F4	F4+F5

F1	F2	F3	F4	F5	F6
0	1	F1+F2	F2+F3	F3+F4	F4+F5

F1	F2	F3	F4	F5	F6
0	1	1	F2+F3	F3+F4	F4+F5

F1	F2	F3	F4	F5	F6
0	1	1	2	F3+F4	F4+F5

F1	F2	F3	F4	F5	F6
0	1	1	2	3	F4+F5

F1	F2	F3	F4	F5	F6
0	1	1	2	3	5

Bottom Up with Tabulation

Tabulation is the opposite of the top-down approach and avoids recursion. In this approach, we solve the problem “bottom-up” (i.e., by solving all the related subproblems first). This is done by filling in a table. Based on the results in the table, the solution to the top/original problem is then computed.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

F1	F2	F3	F4	F5	F6
0	1	F1+F2			

F1	F2	F3	F4	F5	F6
0	1	1			

F1	F2	F3	F4	F5	F6
0	1	1	F2+F3		

F1	F2	F3	F4	F5	F6
0	1	1	2		

F1	F2	F3	F4	F5	F6
0	1	1	2	F3+F4	

F1	F2	F3	F4	F5	F6
0	1	1	2	3	

F1	F2	F3	F4	F5	F6
0	1	1	2	3	F4+F5

F1	F2	F3	F4	F5	F6
0	1	1	2	3	5

Bottom Up with Tabulation

Tabulation is the opposite of the top-down approach and avoids recursion. In this approach, we solve the problem “bottom-up” (i.e., by solving all the related subproblems first). This is done by filling in a table. Based on the results in the table, the solution to the top/original problem is then computed.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

```
def fibonacciTab(n):  
    tb = [0, 1]  
    for i in range(2, n + 1):  
        tb.append(tb[i-1] + tb[i-2])  
    return tb[n-1]
```

Top Down vs Bottom Up

	Top Down	Bottom Up
Easiness	Easy to come up with solution as it is extension of divide and conquer	Difficult to come up with solution
Runtime	Slow	Fast
Space efficiency	Unnecessary use of stack space	Stack is not used
When to use	Need a quick solution	Need an efficient solution

Number Factor

Problem Statement:

- Given N , find the number of ways to express N as a sum of 1, 3, and 4.

Example 1

- $N = 4$
- Number of ways = 4
- Explanation: There are 4 ways we can express $N \rightarrow \{4\}, \{3,1\}, \{1,3\}, \{1,1,1,1\}$

Example 2

- $N = 5$
- Number of ways = 6
- Explanation: There are 6 ways we can express N .
 $\{4,1\}, \{1,4\}, \{3,1,1\}, \{1,3,1\}, \{1,1,3\}, \{1,1,1,1,1\}$

Number Factor

Problem Statement:

- Given N, find the number of ways to express N as a sum of 1, 3, and 4.

NumberFactor(N):

```
if N in (0,1,2) return 1
```

```
if N = 3 return 2
```

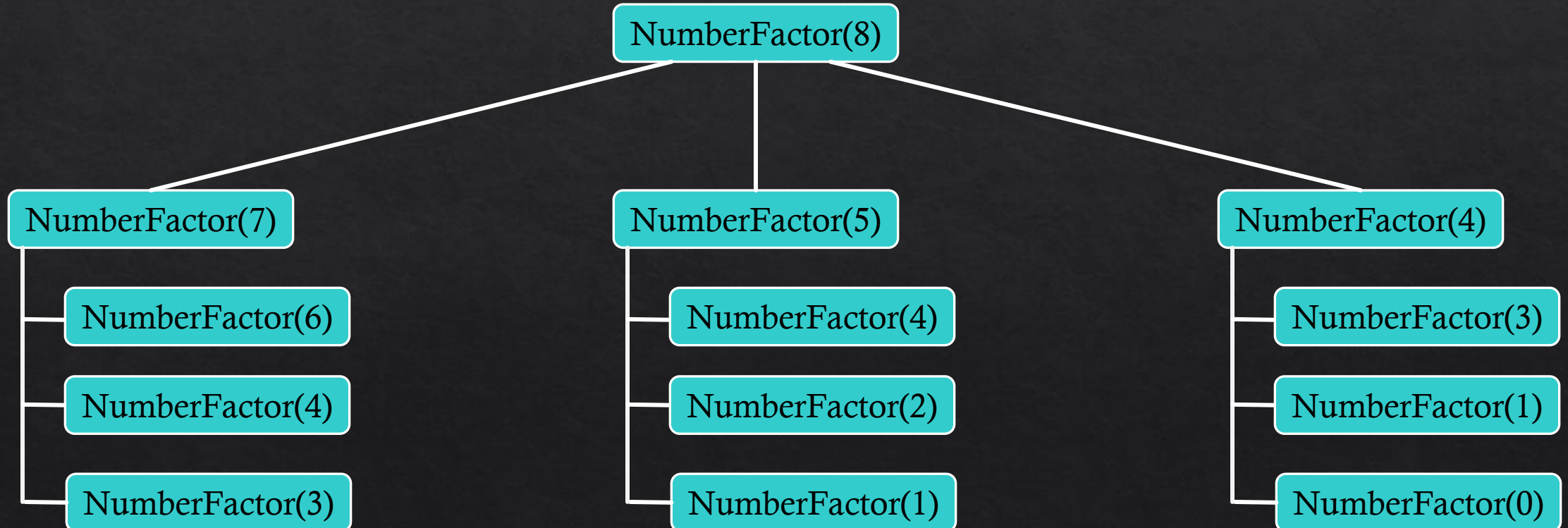
```
else:
```

```
    return NumberFactor(N-1) + NumberFactor(N-3) + NumberFactor(N-4)
```


Number Factor

Problem Statement:

- Given N, find the number of ways to express N as a sum of 1, 3, and 4.



Number Factor

Problem Statement:

- Given N, find the number of ways to express N as a sum of 1, 3, and 4.

NumberFactor(N):

```
if N in (0,1,2) return 1
if N = 3 return 2
else:
    rec1 = NumberFactor(N-1)
    rec2 = NumberFactor(N-3)
    rec3 = NumberFactor(N-4)

    return rec1 + rec2 + rec3
```

NumberFactor(N, dp):

```
if N in (0,1,2) return 1
if N = 3 return 2
elif N in dp return dp[N]
else:
    rec1 = NumberFactor(N-1)
    rec2 = NumberFactor(N-3)
    rec3 = NumberFactor(N-4)
    dp[N] = rec1 + rec2 + rec3
    return dp[N]
```

Number Factor

Problem Statement:

- Given N, find the number of ways to express N as a sum of 1, 3, and 4.

Top Down Approach

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2				NF6+NF4+NF3	1	1	1	2	2+1+1=4	NF4+NF2+NF1	NF5+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2			NF5+NF3+NF2	NF6+NF4+NF3	1	1	1	2	2+1+1=4	4+1+1=6	NF5+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2		NF4+NF2+NF1	NF5+NF3+NF2	NF6+NF4+NF3	1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF1+NF0	NF4+NF2+NF1	NF5+NF3+NF2	NF6+NF4+NF3	1	1	1	2	2+1+1=4	4=1+1=6	6+2+1=9	9+4+2=15

Number Factor

Problem Statement:

- Given N, find the number of ways to express N as a sum of 1, 3, and 4.

Bottom Up Approach

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF2+NF0				1	1	1	2	2+1+1=4	4+1+1=6	NF5+NF3+NF2	

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4				1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	NF4+NF2+NF1			1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)	NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6			1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	9+4+2=15

Number Factor

Problem Statement:

- Given N, find the number of ways to express N as a sum of 1, 3, and 4.

```
def numberFactor(n):  
    tb = [1,1,1,2]  
    for i in range(4, n + 1):  
        tb.append(tb[i-1] + tb[i-3] + tb[i-4])  
    return tb[n]
```